

Programming Assignment: Reinforcement Learning

Deadline

To remain eligible for grade **A** in LAB2, all components required for your target grade and the individual Assignment Learning Journal must be submitted no later than **12 October 2026 at 19:00**. A late submission of any required component results in a maximum LAB2 grade of B.

Individual and pair work

You are encouraged to develop the programs in pairs, but you may also work individually. Groups larger than two are not permitted.

Even when the programs are developed in a pair, each student must submit an **individual Assignment Learning Journal**. You will be assessed individually and must be able to explain the relevant theory, your design decisions, your contribution, and the submitted implementations. Partners may refer to the same implementations and experiments, but each journal must be written individually and reflect the student's own understanding.

What you submit and how it is assessed

This assignment has two required forms of submission:

1. the programs and parameter files required for your target grade, submitted to Kattis; and
2. an individual Assignment Learning Journal, submitted as a PDF in Canvas.

Completion of the required tasks and the Kattis results determine the highest grade range for which the work is eligible. The journal provides evidence of your technical understanding and is used to determine the grade within that range. A TA reviews the journal and code and assigns the final assignment grade. Your journal and code may later be used to prepare questions for your individual oral examination. The oral examination validates your individual understanding and may cap the final LAB2 grade.

This is the final programming assignment in the set. After planning actions by search and modelling hidden processes with HMMs, you will now learn an action policy through interaction and reward.

INTRODUCTION: FISHINGDERBYRL

In FishingDerbyRL, a single diver moves through a grid containing stationary jellyfish and a king fish. The diver can move left, right, down, or up. In the standard scenario, the objective is to catch the king fish while avoiding jellyfish and unnecessary steps. Unlike the search assignment, there is no competing player. Unlike the HMM assignment, the diver observes its own grid position directly.

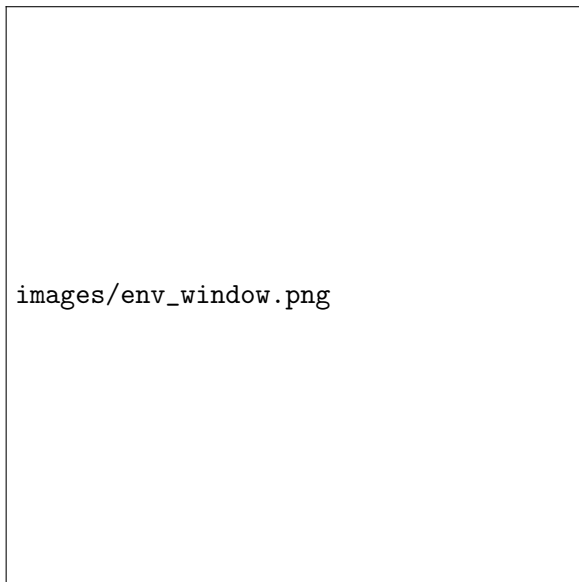


Figure 1: The FishingDerbyRL environment. The diver learns from actions and rewards; fish positions remain fixed within a scenario.

STATES, ACTIONS, AND REWARDS

An MDP comprises a state space $\mathcal{S}$, an action space $\mathcal{A}$, a transition model $P(s' | s, a)$, and a reward rule. The supplied controller represents a state by the diver's position on a 10×10 grid and maintains a table of legal actions at each position. Fish locations and rewards define the scenario and remain fixed across its training episodes. Learn the policy from interactions; do not hardcode routes or use scenario fish coordinates to construct the policy.

The supplied reward list is ordered as

```
[king_reward, jelly_1_reward, ..., jelly_m_reward, living_reward].
```

For a legal move, an ordinary cell gives the living reward, a jellyfish cell gives that jellyfish's reward, and the king cell gives the king reward and ends the episode. **A fish reward replaces the living reward on that step; the two are not added.** Use `rewards[0]` for the king, `rewards[1:-1]` for the jellyfish, and `rewards[-1]` for the living reward. Choose legal moves; the supplied environment penalizes an illegal move by -100 without moving the diver.

An episode ends when the king is caught or the configured step limit is reached (100 steps in the assignment tests). The next training episode starts from the configured initial position. The learning objective uses discounted return,

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1},$$

whereas the episode's printed total reward is the *undiscounted* sum. These quantities need not rank policies identically. The supplied policy is a position-to-action mapping; the step limit is also a stopping condition, rather than an extra coordinate in its Q-table.

GETTING STARTED AND GRADE ELIGIBILITY

Download `r11.zip` through `r15.zip` from the corresponding RL problems on the course Kattis assignment page linked in Canvas. Extract each into a separate directory. Follow the supplied installation instructions, then run from the directory containing `main.py`:

```
python main.py settings.yml
```

Keep the file named `player.py` for local runs; make the numbered copy indicated below for submission. Carry your completed methods forward into later starters, preserving their interfaces. A later starter does not automatically contain your earlier work.

Set `player_type` to `random` or `ai_rl` as appropriate. Use `headless: true` for faster local experiments; set it to `false` to enable the graphical application. The separate `visualize_exploration` setting controls display of training. Record seeds in local experiments and use `numpy.random` for random choices, consistently with the supplied evaluation setup. Do not assume that repeated unseeded runs are comparable.

Preserve the provided function names, arguments, and message format. Other skeleton files may be changed for local investigation, but Kattis supplies its own versions. Local changes to `settings.yml` are not submitted: use the parameter files for RL-3 and RL-5. Do not read standard input or add standard-output messages in submitted player code; use standard error for debugging. Preserve the skeleton's communication loop.

CUMULATIVE REQUIREMENTS

Problem	Files submitted to Kattis	Required score
RL-1	<code>player_1.py</code>	10
RL-2	<code>player_2.py</code>	25
RL-3	<code>player_3.py</code> , <code>student_3_2_1.py</code> , <code>student_3_2_2.py</code>	10
RL-4	<code>player_4.py</code>	23
RL-5	<code>player_5.py</code> , <code>student_5.py</code>	26

E–D: complete Sections 1–3 and meet the RL-1, RL-2, and RL-3 thresholds. **C:** also complete Section 4 and meet the RL-4 threshold. **B–A:** also complete Section 5 and meet the RL-5 threshold. All levels require an individual journal. These are separate problem scores, not a combined score across RL-1–RL-5.

The assignment grade is the highest grade for which the implementation and journal together provide sufficient evidence. A Kattis threshold establishes eligibility, not an automatic grade. The distinction between E and D, and between B and A, depends on the journal's evidence of understanding, decisions, and learning. Grade A also requires on-time submission.

Questions **Q1–Q6** are assignment questions. Answer all six in **J2** of your individual Assignment Learning Journal. They concern the core tasks required at every grade level. Document the exploration and advanced investigations for higher grades in J3 and J4.

1 GRADE LEVELS E AND D: RANDOM AGENTS

1.1 OVERVIEW OF THE ENVIRONMENT

Use `rl1.zip`. Inspect `FishingDerbyRLApp` in `app_manager.py`, especially its state/action initialization, `step`, and reward computation. Inspect `init_states`, `init_actions`, and `allowed_movements` in `player.py`. The naming of the state-index dictionaries can be confusing: check the key and value types rather than inferring their direction from their names.

Q1. Describe the state representation, number of table states, and available actions in the supplied grid world. How do legal actions depend on position? What does the agent observe after a move, what remains fixed across episodes, and what ends an episode?

1.2 IMPLEMENT A RANDOM AGENT

Implement `PlayerControllerRandom.random_agent()` in `player.py`. Fill the code regions marked EX. 1.2. Your agent must:

- initialize a state-action count table;
- sample an action uniformly from the legal actions at its current state;
- increment that state-action count, send the action through the supplied communication loop, and receive the next state, reward, and episode-end flag;
- handle episode resets and stop within `self.episode_max` episodes.

Return the count table in the format expected by `get_policy`. The final policy chooses a most frequently sampled action for each state; it is distinct from the random behaviour used during training. Ensure that final policy actions are legal, including at unvisited boundary states and when counts tie. You may adjust policy extraction in `player.py` while retaining its interface.

Use the configured episode limit as an upper bound. The placeholder loop uses a zero-based counter: check its condition to avoid running one extra episode. Verify the communication loop with a small episode budget before running longer experiments.

Q2. Why is a random agent useful as a baseline? Explain the difference between its training behaviour and the final count-based policy. Give one limitation of judging the quality of a policy only from rewards collected during training.

SUBMISSION AND EVIDENCE

Submit `player_1.py` to **RL1 Random Agents**; obtain at least **10 points**. Keep a short local trace showing a valid action, reward, and episode reset. Use this when checking your implementation; do not paste a full trace into the journal.

2 GRADE LEVELS E AND D: Q-LEARNING

Use `r12.zip` and implement `PlayerControllerRL.q_learning()`. Set `player_type: ai_rl`. The controller returns a Q-table from which `get_policy` extracts a greedy policy. The starter's markers EX. 2.1, EX. 2.2, and EX. 2.3 identify initialization, updates, and convergence, respectively.

2.1 INITIALIZE AND UPDATE THE Q-TABLE

Create a floating-point table with one row per state and one column per action, following the starter's initialization instructions. The starter marks illegal state-action entries with `np.nan`. Exclude them from action selection, maximization, and convergence statistics. Begin with random legal action selection, then compare it with greedy selection among legal actions.

Implement the Q-learning update using `self.alpha` and `self.gamma`:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [y_t - Q(s_t, a_t)], \quad y_t = r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_{t+1})} Q(s_{t+1}, a').$$

The displayed target applies to a continuing transition. At a terminal transition there is no future return, so use $y_t = r_{t+1}$. For this assignment's episodic implementation, treat the supplied `end_episode` flag as the end of the sampled return, including the step limit. Do not update using the reset position as though it were a continuation of the preceding episode.

Q3. Explain the Q-learning target and the role of each term in the update. Show one numerical update of your choice. How does your code handle episode-ending transitions and illegal actions? Refer to the relevant method.

Test greedy action selection using the same episode budget and several recorded seeds. In the standard local reward configuration, a best route gives an undiscounted episode reward of 11. This value is scenario-specific, not a universal maximum for all reward settings. Explain why a greedy learning agent can fail to find that route.

2.2 STOPPING TRAINING

Combine the maximum episode budget with a convergence statistic. After an episode, compare the current Q-table with its saved previous version. Compute the *mean absolute change* over legal entries, not the absolute value of the mean signed change, whose positive and negative terms can cancel. Use the configured `threshold` (normally 10^{-6}) and update the saved table at the correct point.

A small change is a practical stopping heuristic, not proof of optimality: unvisited actions may retain inaccurate values. Check how early stopping behaves under your exploration strategy. If your starter uses `np.infty` and your NumPy version does not provide it, use `float('inf')` in your player file.

Q4. State your convergence statistic and stopping rule. Why does a small Q-table change not establish that the policy is optimal? Give a concrete failure case involving exploration or unvisited actions, and explain how you checked for it.

Submit `player_2.py` to **RL2 Q-learning**; obtain at least **25 points** for eligibility in the E-D range, together with the other core requirements.

3 GRADE LEVELS E AND D: HYPERPARAMETERS AND ENVIRONMENT

Use `r13.zip` and carry forward your Q-learning implementation. This part separates parameters of the learning algorithm from the rewards defining the task.

3.1 HYPERPARAMETERS

Investigate the learning rate α and discount factor γ . The learning rate controls the size of each update; the discount factor controls the weight of future rewards and can change the policy preferred by the learning objective. It is not an exploration parameter.

Use controlled local comparisons to investigate: (1) little or no learning; (2) fast learning with variable outcomes; (3) more consistent outcomes with high long-term return; and (4) variable outcomes with high long-term return. These are behaviours to investigate, not four parameter intervals guaranteed to exist in every scenario. For the no-learning control you may set $\alpha = 0$; for actual learning use $0 < \alpha \leq 1$. Use $0 \leq \gamma \leq 1$ and explain your choice.

Keep the scenario, initialization procedure, episode budget, and action-selection strategy fixed when isolating a hyperparameter's effect. Use the same small set of seeds for each alternative and say what you mean by variability (for example, final policy return across seeds). Summarize the most informative comparison in J3.

Q5. Distinguish the effects of α and γ . Describe one controlled comparison you performed, what you held fixed, and what changed. Does your evidence support your expectation? Explain a limitation of the comparison.

3.2 CHANGING THE REWARD STRUCTURE

Edit `student_3_2_1.py` and `student_3_2_2.py` to obtain these behaviours:

1. avoid catching any fish, including the king, until the episode reaches its step limit;
2. take a shortest route to catch the king.

Change the reward values to make each behaviour desirable. Use the reward indexing defined in the introduction. Keep the other learning parameters fixed at values already validated with your RL-2 agent so that the comparison isolates the reward change. The supplied parameter templates contain $\alpha = 0$; replace such placeholders with your working settings before testing or submitting. Preserve every parameter variable name.

To test locally, copy the parameter values into `settings.yml`; the local application does not automatically import the student parameter files. Run your agent and inspect the resulting policy. For the additional scenario in `3_2_3.yml`, use its values in a complete local settings file if you want to simulate it.

Q6. Using the positions and rewards in `3_2_3.yml`, determine the maximum undiscounted episode return and justify it with a route or step count. Apply the actual reward rule on the king-catching step. Explain why this return differs from a discounted return with $\gamma < 1$.

Submit `player_3.py`, `student_3_2_1.py`, and `student_3_2_2.py` to **RL3 Hyperparameters and Environment**; obtain at least **10 points**.

4 GRADE LEVEL C: EXPLORATION VERSUS EXPLOITATION

First meet all E–D requirements. Use `rl4.zip` and carry forward your Q-learning implementation. In `player.py`, implement the **module-level function** `epsilon_greedy` and `ScheduleLinear.value`. The epsilon-greedy function is not a method of `PlayerControllerRL`. Preserve its supplied signature: the evaluator can call it directly.

4.1 CONSTANT EPSILON-GREEDY

For `eps_type='constant'`, the starter uses `epsilon_final` as the constant exploration probability. With probability ϵ , select uniformly from the supplied legal action indices; otherwise, select a maximizing legal action from the current state's Q-values. Support $0 \leq \epsilon \leq 1$, including the endpoint cases of purely greedy and purely random selection.

Use the action indices passed as `all_actions`; they need not form a consecutive list beginning at zero. Never select an action whose Q-value marks it as illegal. Return the selected action index expected by the caller. Integrate the function into action selection in `q_learning()`.

4.2 LINEAR ANNEALING

Implement `ScheduleLinear.value(t)` so that exploration falls from an initial value to a final value over a specified number of training steps. For a positive duration T , use

$$\epsilon_t = \epsilon_{\text{initial}} + (\epsilon_{\text{final}} - \epsilon_{\text{initial}}) \min\left(1, \max\left(0, \frac{t}{T}\right)\right).$$

After the schedule finishes, retain the final value. Handle a zero duration explicitly by returning the final value. In the linear branch of `epsilon_greedy`, use the schedule with the supplied arguments.

The function defaults specify a decrease from 1.0 to 0.2 over 10,000 steps. Local settings can override these values: inspect them rather than assuming that the defaults are active. Count total *training steps across episodes*, not episodes, and do not restart the schedule at every episode.

4.3 REQUIRED COMPARISON FOR C OR ABOVE

Compare greedy selection, constant epsilon-greedy, and linear annealing on the same local scenario and the same set of at least three recorded seeds. Keep the Q-table initialization procedure, learning parameters, and maximum training budget fixed. State how you handle early stopping so that its effect on exploration is visible.

Report final greedy-policy performance separately from exploratory training returns. Include king-capture success, final-policy return, and the number of training episodes used. Explain whether each strategy found a good policy reliably and identify a limitation of the comparison. Summarize the main evidence in J3; extensive logs are not required.

Submit `player_4.py` to **RL4 Exploration vs Exploitation**; obtain at least **23 points**. The journal must show C-level understanding of exploration, scheduling, and their interaction with the stopping rule.

5 GRADE LEVELS B AND A: ESCAPING A SUBOPTIMAL POLICY

First meet the E–D and C requirements. Use `r15.zip`, carry forward your completed methods, and use its `settings.yml` as the local scenario. The required parameter file is `student_5.py`; no separate `5_student.yml` is needed.

Your task is to learn a strong policy reliably across different seeds and scenarios within a limited training budget. A route that initially appears attractive can keep an agent from exploring a better one. Improve your exploration strategy, learning parameters, or stopping criterion, and explain the mechanism by which the change should help.

5.1 BUDGET AND INTERFACE

Use at most **2,000 training episodes**, each of at most **100 steps**, in the RL-5 evaluation. Always respect `self.episode_max` as supplied by the environment; do not overwrite it to increase the budget. The downloaded local settings default to 1,000 episodes. You may set the local budget to 2,000 to investigate the full RL-5 allowance. Correct any off-by-one loop condition in your player code.

Set the named variables in `student_5.py`: `alpha`, `gamma`, `epsilon_initial`, `epsilon_final`, `annealing_timesteps`, and `threshold`. Use the corresponding settings fields in your implementation. Local experiments require these values to be copied into `settings.yml`. Do not add reward or episode-limit overrides to the submission parameter file.

The evaluator obtains a final policy from your learned Q-table. Evaluate that policy explicitly: a high exploratory episode return alone does not show that the extracted greedy policy is equally good. If retaining a best-so-far candidate, retain the table or policy that actually produced the measured result. Do not equate a repeated best observed return with proof of global optimality.

5.2 REQUIRED INVESTIGATION FOR B OR A

Build on the exploration comparison from Section 4. Compare your final approach with a reasonable alternative using the same scenario and at least five recorded seeds per approach. State the initialization, parameter settings, maximum budget, and stopping criterion. Record final-policy king-capture success, undiscounted return, steps, and training episodes. Report failures as well as successful runs.

Use J4 to analyze why the method should escape a suboptimal policy, what the evidence supports, and where it may fail. For grade A, critically compare the alternatives with concrete evidence and justify the choice under the training constraint. Repeatedly tuning constants until a Kattis threshold is reached is not by itself sufficient evidence for B or A.

5.3 SUBMISSION AND GRADE REQUIREMENT

Submit `player_5.py` and `student_5.py` to **RL5 ESCAPING a sub-optimal**. Obtain at least **26 points**. Together with the cumulative requirements, this establishes eligibility for B or A. The journal distinguishes B from A; A also requires on-time submission.

RL-5 evaluates three scenarios. Its score reflects policy performance and training efficiency; the required score is the aggregate reported by Kattis. The value 11 is a reference for the standard local scenario, not the target return for every hidden scenario. Your experiments should support robustness, not a claim of guaranteed success for every possible seed.

6 ASSIGNMENT LEARNING JOURNAL

The Assignment Learning Journal is an individual, concise account of what you implemented, why you made important decisions, and what you learned. It is not a diary or a full technical report. Connect explanations to your submitted code, experiments, and observations.

Completion of the required tasks and the Kattis results determine the highest eligible grade range. The journal provides evidence of individual understanding within that range. A TA reviews the journal and code and assigns the final grade. The journal and code may also be used to prepare questions for the individual oral examination.

6.1 FORMAT AND SUBMISSION

Use the headings **J0–J6** and submit the journal as a PDF in Canvas. Most journals will be approximately **800–1200 words**. The maximum is **1500 words**, excluding the question prompts, J0, and J6. Students aiming for E or D will normally be closer to the lower end; students aiming for C, B, or A may need the upper end.

Short paragraphs and bullet points are welcome. Do not paste source code or screenshots; refer to files, classes, or functions by name. One small results table is permitted when it makes an empirical comparison clearer. Count explanatory text in the table within the word limit. Summarize evidence rather than reproducing every experimental run.

If you worked with a partner, the implementations and experiments may be shared, but the journal must be written individually. Shared facts may overlap; your explanations, contribution, and learning must be your own. Assessment concerns technical understanding, not writing style.

J0 – SUBMISSION RECORD

Give your name and KTH email, your partner’s name (or “none”), and the grade you are aiming for. List the Kattis submission identifier and score for RL-1, RL-2, and RL-3. If aiming for C or above, also list RL-4; if aiming for B or A, also list RL-5. Identify the parameter files submitted with RL-3 and RL-5 where applicable.

J1 – SOLUTION OVERVIEW (ABOUT 100–150 WORDS)

Explain how your agent moves from environment interactions to a Q-table and a final policy. Name the important files, classes, or functions and distinguish provided code from what you implemented or substantially changed. Explain how the random baseline and learning agent relate. If aiming for C or above, also summarize the role of exploration and its schedule.

J2 – ASSIGNMENT QUESTIONS

Answer **Q1–Q6**, normally in about 40–80 words each. Use a small calculation, formula, trace, or code reference when helpful. Explain in your own words rather than only repeating a definition. These answers cover the common core; do not add separate answers to unnumbered instructions that are already addressed through J3 or J4.

J3 – DECISIONS AND EVIDENCE

Describe **two important decisions** concerning, for example, state-action representation, initialization, rewards, learning parameters, convergence, exploration, policy extraction, or testing. For each, give the goal, alternatives considered, choice and reason, code location, evidence, and a trade-off or limitation. Evidence may be a controlled comparison, local test, trace, measurement, Kattis result, or failure case.

Include the most informative evidence from the hyperparameter and reward investigations in Section 3. If aiming for C or above, also summarize the required exploration comparison from Section 4 and connect at least one decision to its results. At least one decision must include a revised approach, unexpected behaviour, important boundary case, remaining limitation, or meaningful alternative you evaluated. Refer back to J2 where appropriate instead of repeating it.

J4 – ADVANCED ANALYSIS (REQUIRED WHEN AIMING FOR B OR A)

Build on one of the decisions involved in the RL-5 agent. Explain how it works, why it should help, evidence of its effect, and a limitation or trade-off. Describe a reasonable alternative. Include the comparison required in Section 5, separating training behaviour from final-policy evaluation and showing variability across seeds.

For grade A, critically compare the alternatives using concrete evidence and justify your choice under the assignment’s constraints. If aiming for E, D, or C, retain the heading and write “Not applicable to my target grade.”

J5 – INDIVIDUAL LEARNING AND CONTRIBUTION (ABOUT 100–150 WORDS)

State what you personally contributed. Describe the most important thing you understand now that you did not understand before, the hardest issue you resolved, and one limitation or open question that remains. As this is the final assignment in the set, use one brief example to connect your learning to Search or HMM: for example, planning with a model versus learning through interaction, or estimating a model versus learning action values. Keep this within J5; no separate course-wide report is required.

J6 – USE OF AI TOOLS

State whether and how you used generative AI or other automated assistance in this assignment, both for the code and this journal. Identify what it was used for and explain how you verified and understood any resulting suggestions or code. If you did not use such tools, state this explicitly. Use of these tools is permitted as support, but you remain responsible for all submitted code and text and must be able to explain them during the individual oral examination. Do not paste complete prompts or conversations.

BEFORE SUBMITTING

Check that each required Kattis problem has the correct numbered player file and any required parameter files, that your recorded scores meet your target range, and that your individual journal contains J0–J6 and answers Q1–Q6. Submit the journal and all required Kattis components by the assignment deadline.